Deep Learning Statistical Arbitrage: An Empirical Study of Neural Trading Policies on Equity Residuals

Hadis Surya Al Amin[1]

[1]Project report – based on the framework of Guijarro-Ordonez, Pelger, and Zanotti (2021)

June 10, 2026

Abstract

We study data-driven statistical arbitrage in U.S. equities by learning trading policies directly from *residual* return series. Residuals are extracted with three families of factor models—IPCA, PCA, and the Fama–French five-factor model—and a neural trading policy is trained end-to-end to maximize an out-of-sample Sharpe ratio. We evaluate a convolution–transformer architecture with a graph component (`CNNTransformer`) against feed-forward baselines (`RawFFN`, `FourierFFN`, `OUFFN`). This report describes the data, methodology, and empirical results, including risk-adjusted performance and turnover across factor models.

1 Introduction

Classical statistical arbitrage exploits temporary mispricings in the residual component of asset returns—the part left after systematic (factor) exposure is removed. Rather than assuming a parametric mean-reverting process (e.g. an Ornstein–Uhlenbeck model) and hand-tuning entry/exit thresholds, we follow the *Deep Learning Statistical Arbitrage* framework and learn the trading policy directly from data. The policy maps a window of recent residual behavior to a portfolio weight, and is optimized to maximize a risk-adjusted objective on out-of-sample (OOS) data via rolling retraining.

2 Data and Residual Construction

The investable universe is defined by a market-capitalization cap proportion (`cap_proportion` = 0.01). For each asset we form a residual time series by removing common factors using one of three factor models:

- **IPCA** — Instrumented Principal Component Analysis (here 5 factors),
- **PCA** — rolling-window principal component analysis (5 factors),
- **Fama–French** — the daily five-factor model.

Daily trading dates are aligned to the Fama–French research data over the OOS period (1998–2017). When `use_residual_weights` is enabled, the residual composition matrix is used to translate residual-space positions into asset-space weights for realistic turnover and short-proportion accounting.

3 Methodology

We compare four trading-policy architectures, all consuming a lookback window of preprocessed residuals (cumulative-sum normalized): `RawFFN` (feed-forward on raw windows), `FourierFFN` (Fourier features), `OUFFN` (Ornstein–Uhlenbeck-inspired layer), and `CNNTransformer`. The remainder of this report focuses on the `CNNTransformer` family and, in particular, its graph-augmented variant `CNNTransformer+GNN`.

3.1 The `CNNTransformer+GNN` architecture

The policy maps, for each trading day, the recent residual paths of all tradable assets to a vector of portfolio weights. Let N be the number of active residuals on a given day and L the lookback ($L = 30$). Processing flows through four stages (implemented in `models/CNNTransformer.py`).

(1) Input and preprocessing. For each residual i and day t we form a window of the *cumulative* residual return over the last L days (`preprocess_cumsum`): $x_{i,t} \in \mathbb{R}^L$. A boolean validity mask selects residuals with no missing observation inside the window, so the cross-section N varies over time without inducing look-ahead bias.

(2) Temporal convolution (per residual). Each window is passed, independently across residuals, through a 1-D convolutional block: two causal (left-zero-padded) `Conv1d` layers with ReLU and instance normalization, a residual skip connection, mapping $1 \to 8$ channels (`filter_numbers` $= [1, 8]$, `filter_size`$= 2$). Output: a feature map in $\mathbb{R}^{8\times L}$ per residual. This stage extracts local mean-reversion/momentum patterns in each residual's own path.

(3) Graph layer (cross-sectional, the GNN extension). At each time step the convolved features of all valid residuals are treated as nodes of a *fully connected* graph. Each node is updated from the concatenation of its own features and the *mean of its neighbors'* features, followed by a linear map $\mathbb{R}^{2\cdot 8} \to \mathbb{R}^8$ and ReLU, with masking so that padded (invalid) residuals contribute nothing:

$$h_i' \;=\; \mathrm{ReLU}\Big(W\,\big[\,h_i \,\|\, \tfrac{1}{|\mathcal{N}_i|}\sum_{j\in\mathcal{N}_i} h_j\,\big]\Big), \qquad \mathcal{N}_i = \{\text{all other valid residuals}\}. \tag{1}$$

This is the only component that shares information *across* the cross-section; disabling it (`use_gnn=False`) recovers the paper-baseline `CNN+Transformer`. With the GNN enabled the per-node feature dimension becomes `gnn_hidden_dim`$= 8$ (1 layer).

(4) Transformer and output head. The (per-residual) feature sequence over the L time steps is fed to a `TransformerEncoderLayer` (`attention_heads`$= 4$, feed-forward width $= 2\times$ model dimension, dropout 0.25) which models temporal dependencies via self-attention. The representation at the last time step is passed through a linear head $\mathbb{R}^d \to \mathbb{R}$, yielding one raw weight per residual. Raw weights are normalized to unit gross exposure, $w_t \leftarrow w_t / \sum_i |w_{i,t}|$, giving the dollar-neutral-style portfolio for day t.

3.2 Notation

We fix the following symbols for the algorithm below.

Symbol	Meaning
$\epsilon \in \mathbb{R}^{T\times N}$	residual-return matrix: T days $\times$ N residuals (one per asset)
$\epsilon_{i,t}$	residual return of asset i on day t
$L = 30$	lookback (days of history fed to the model)
$x_{i,t} \in \mathbb{R}^L$	preprocessed (cumulative-return) window for asset i ending at day t
$m_{i,t} \in \{0, 1\}$	validity mask (1 if asset i has a complete window at t)
$f_\theta(\cdot)$	the `CNNTransformer+GNN` network with parameters θ
$w_{i,t}$	portfolio weight on asset i for day t (model output)
$r_t = \sum_i w_{i,t}\,\epsilon_{i,t}$	realized portfolio return on day t
$L_{\text{tr}} = 1000,\ f = 125$	training-window length and retrain frequency (days)

3.3 Forward pass: from residual windows to portfolio weights

For a single trading day t, the policy turns the recent residual paths of all valid assets into one portfolio. There is no label; the "prediction" *is* the portfolio. The four stages of f_θ map the dimensions as follows (batched over the N assets):

(1) windows	$x_{\cdot,t} \in \mathbb{R}^{N\times L}$	cumulative-return window per asset
(2) CNN	$\xrightarrow{\text{1-D conv, 1}\to\text{8 ch}} H \in \mathbb{R}^{N\times 8\times L}$	local temporal features, per asset
(3) GNN	$\xrightarrow{h_i \mapsto \mathrm{ReLU}(W[h_i \| \bar{h}_{\mathcal{N}_i}])} H' \in \mathbb{R}^{N\times 8\times L}$	cross-sectional mixing (this stage = the extension)
(4) Transformer + head	$\xrightarrow{\text{self-attn over } L\text{, then linear}} \tilde{w}_{\cdot,t} \in \mathbb{R}^N$	one raw weight per asset

followed by gross-exposure normalization $w_{i,t} = \tilde{w}_{i,t} / \sum_j |\tilde{w}_{j,t}|$. In stage (3), $\bar{h}_{\mathcal{N}_i}$ is the mean of all *other* valid assets' features; setting `use_gnn=False` removes this stage entirely, so the baseline weight depends only on asset i's own window.

3.4 Training objective

Given a block of days, the network is trained to make the resulting return series $\{r_t\}$ have the highest possible Sharpe ratio, by minimizing

$$\mathcal{L}(\theta) \;=\; -\,\frac{\mathrm{mean}_t(r_t)}{\mathrm{std}_t(r_t)}, \qquad r_t = \sum_i w_{i,t}\,\epsilon_{i,t} \;-\; (\text{trading costs}). \tag{2}$$

Gradients flow from this Sharpe loss all the way back through the weights, the transformer, the GNN, and the convolution—hence *end-to-end*.

3.5 Rolling train/test loop

To keep every evaluated day out-of-sample, training and testing slide forward in blocks of $f = 125$ days; a fresh model is trained on the preceding $L_{\text{tr}} = 1000$ days for each block (Algorithm 1).

Algorithm 1 Rolling out-of-sample evaluation of the policy

Require: residual matrix $\epsilon \in \mathbb{R}^{T \times N}$; $L_{\text{tr}} = 1000$; $f = 125$; $L = 30$

for $k = 0, 1, \ldots, \lfloor (T - L_{\text{tr}})/f \rfloor$ **do** ▷ 31 subperiods
  $\text{Train}_k \leftarrow \epsilon[\, kf \,:\, kf + L_{\text{tr}} \,]$ ▷ 1000-day training block
  $\text{Test}_k \leftarrow \epsilon[\, kf + L_{\text{tr}} - L \,:\, kf + L_{\text{tr}} + f \,]$ ▷ next 125 days (strictly later)
  initialize fresh parameters θ_k ▷ rolling retrain
  for epoch $= 1, \ldots, 100$ **do** ▷ train on Train_k
    for each minibatch of 32 days **do**
      $w \leftarrow \text{normalize}\big(f_{\theta_k}(\text{windows})\big)$; $\quad r_t \leftarrow \sum_i w_{i,t}\, \epsilon_{i,t}$
      $\theta_k \leftarrow$ Adam step on $\mathcal{L} = -\,\text{mean}(r)/\,\text{std}(r)$
    end for
  end for
  save checkpoint θ_k; run f_{θ_k} on Test_k ▷ out-of-sample
  record OOS returns r_t, turnover, short proportion for this block
end for
return concatenated OOS return series and metrics over all 3,781 test days

With $T = 4781$ and $f = 125$ this gives 31 retrained subperiods (checkpoints `subperiod0`–`subperiod30`). Crucially, Test_k always lies strictly after Train_k, so no future information leaks into the reported performance.

3.6 Evaluation metrics

For each (model, factor model) pair we report the annualized Sharpe ratio (daily Sharpe $\times\sqrt{252}$), mean daily return, return standard deviation, turnover, and short proportion.

4 Experimental Setup

All experiments train the `CNNTransformer` on the `IPCA5`, `PCA5`, and `FamaFrench5` residual sets under the rolling protocol of Algorithm 1, in two configurations that differ *only* in the GNN component: the **paper baseline** (`CNN+Transformer`, `use_gnn=False`) and the **GNN extension** (`CNN+Transformer+GNN`, `use_gnn=True`). This isolates the marginal effect of cross-sectional message passing. The shared hyperparameters are listed in Table 1.

Table 1: Training and architecture hyperparameters (from `configs/cnntransformer-full.yaml`).

Data / protocol		*Architecture*	
Lookback L	30 days	Conv filters	$[1, 8]$, size 2
Training window	1000 days	Conv normalization	instance norm
Retrain frequency	125 days	Attention heads	4
Rolling retrain	yes	FF width factor	2 ($\Rightarrow$ 16)
Subperiods	31	Dropout	0.25
OOS days	3,781	GNN type	fully connected
Cap proportion	0.01	GNN hidden / layers	8 / 1
Optimization		*Costs / accounting*	
Optimizer	Adam	Transaction cost	0
Learning rate	10^{-3}	Holding cost	0
Batch size	32	Residual weights	`False`
Max epochs	100	Objective	Sharpe

5 Results

Table 2 summarizes risk-adjusted performance across factor models, sorted by annualized Sharpe ratio. Figure 1 shows the corresponding out-of-sample equity curves (cumulative return, log scale).

Table 2: Out-of-sample performance by trading policy and factor model.

Factor model	Architecture	Sharpe (ann.)	Mean ret.	Std.	Turnover	Short prop.
PCA5	CNN+Transformer	5.014	0.000	0.001	1.066	0.364
PCA5	CNN+Transformer+GNN	4.912	0.000	0.001	1.045	0.347
IPCA5	CNN+Transformer	4.204	0.000	0.001	1.108	0.508
IPCA5	CNN+Transformer+GNN	3.900	0.000	0.001	1.024	0.479
FamaFrench5	CNN+Transformer	3.080	0.000	0.001	1.101	0.451
FamaFrench5	CNN+Transformer+GNN	2.969	0.000	0.001	1.049	0.480

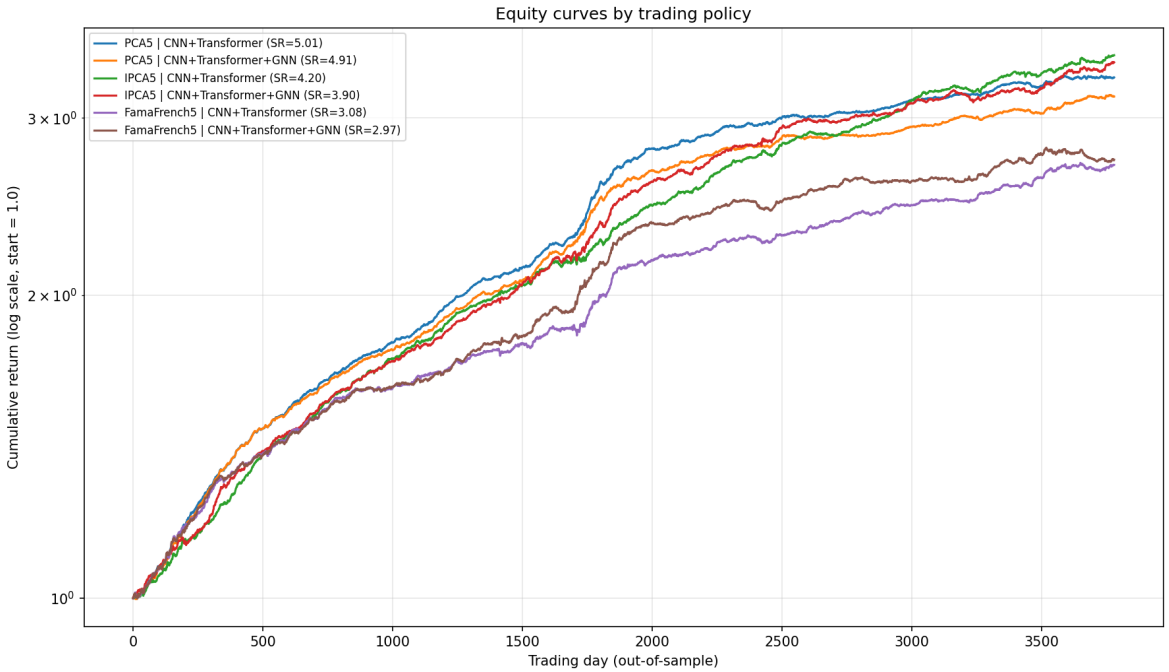

Figure 1: Out-of-sample cumulative returns (equity curves) for each trading policy, log scale, normalized to a starting value of 1.0.

5.1 Discussion

Three findings stand out from Table 2, computed over 3,781 out-of-sample trading days with zero transaction and holding costs and without residual-weight adjustment (`use_residual_weights = False`).

Factor model matters most. The choice of residual-generating factor model dominates performance. **PCA** residuals deliver the strongest risk-adjusted returns (annualized Sharpe ≈ 5.0), followed by **IPCA** (≈ 4.2) and **Fama–French** (≈ 3.1). The PCA advantage comes not from higher mean returns—IPCA actually has a marginally higher mean daily return—but from substantially lower volatility, yielding a cleaner, more mean-reverting residual signal for the policy to exploit.

The GNN extension does not help here. Adding the graph component (`CNN+Transformer+GNN`) slightly *reduces* the Sharpe ratio for all three factor models: PCA 5.01 $\rightarrow$ 4.91, IPCA 4.20 $\rightarrow$ 3.90, and Fama–French 3.08 $\rightarrow$ 2.97. The differences are small and the GNN does modestly lower turnover (e.g. IPCA 1.11 $\rightarrow$ 1.02), but on this universe and configuration the cross-sectional graph information provides no out-of-sample edge over the plain convolution–transformer policy. This is a useful negative result: the extra parameters do not pay for themselves before costs.

Why might the GNN add little? (interpretation). A plausible explanation is that the factor model has already removed the dominant cross-sectional comovement among assets: residuals are, by construction, the part of returns left *after* common factor structure is stripped out, and are therefore close to idiosyncratic and only weakly related across the cross-section. If little exploitable cross-sectional structure remains in the residual series, the GNN's message-passing across residuals has correspondingly little to add over a policy that already models each residual's own temporal

dynamics. We stress that this is an interpretation, not a demonstrated mechanism: the performance margin is small (and within plausible seed-to-seed variation), the comparison is conducted only in the residual-space, cost-free regime, and a single GNN configuration (a one-layer fully connected graph) was tested. The cross-sectional benefit of a graph component could still emerge under realistic position accounting (`use_residual_weights = True`), under transaction costs, or with a richer graph (e.g. sector- or correlation-based edges).

Turnover and costs. All policies trade aggressively, with average daily turnover near 1.0–1.1 (the book is effectively rebuilt each day) and short proportions of 0.35–0.51. Because these results assume zero transaction costs, the headline Sharpe ratios are gross figures; the high turnover implies the ranking could compress materially once realistic costs are introduced, which is the natural next experiment.

6 Conclusion

We presented an end-to-end deep learning approach to statistical arbitrage on equity residuals and benchmarked several neural trading-policy architectures across IPCA, PCA, and Fama–French factor models under a rolling out-of-sample protocol.

Reproducibility

Results are produced with `run_train_test.py` using the configuration files in `configs/`. To regenerate the table and figure in this report:

```
python3 tools/analyze_results.py            # all models
python3 tools/analyze_results.py -m CNNTransformer
```

To match the published paper, set `use_residual_weights: True` in the config.

References

[1] J. Guijarro-Ordonez, M. Pelger, and G. Zanotti. *Deep Learning Statistical Arbitrage*, 2021. https://arxiv.org/abs/2106.04028.